

Cadence Manual

June 10, 2026

Contents

1	Setup and Installation	3
1.1	Repository Structure	3
1.2	Configuration	3
1.2.1	global.cfg	3
1.2.2	waltzer_nuv.cfg, waltzer_vis.cfg, waltzer_nir.cfg	4
1.2.3	excel_mapping.cfg	4
1.2.4	Constants	4
1.2.5	parameters.txt / User Config	4
1.3	Data Files	4
1.3.1	Effective Area Files	4
1.3.2	Cross-Dispersion Spread Profile	5
1.3.3	NIR PSF File	5
2	Running the Simulator	6
2.1	Excel Target Catalogue	6
2.2	Gaia Helper Scripts	6
2.3	Gaia Lookups	6
3	Tests	8
3.1	test_flux_pipeline.py	8
3.2	test_detector_pipeline.py	8
3.3	Regenerating Snapshots	8
4	Extending the Simulator / Performance Considerations	9
4.1	Adding Star and Planet Parameters	9
4.2	Extending Channel and Global or User Configuration	9
4.3	Adding Additional ISM Absorption Lines	9
5	Caches and Performance Improvements	10
5.1	ISM Absorption Optimisation / Voigt Profile Cache	10
5.1.1	Voigt Profile Cache	10
5.1.2	Adding Another ISM Absorption Line	11
5.2	Global Configuration Cache	11
5.3	User Configuration Cache	11
5.4	Stellar Model Caches	12

5.5 Unred Curve Cache	12
---------------------------------	----

1. Setup and Installation

Clone the repository and install all required packages (easiest, needed for tests on github):

```
pip install -r tests/config/requirements.txt
```

Some features require network access at runtime, specifically Gaia archive lookups and SIMBAD name resolution via `astroquery`.

Required packages: `numpy`, `scipy`, `matplotlib`, `astropy`, `astroquery`, `openpyxl`, `pandas`, `pytest`.

1.1. Repository Structure

- `configs/` — global and per-channel instrument configuration files:
 - `global.cfg` — global simulation settings
 - `waltzer_nuv.cfg`, `waltzer_vis.cfg`, `waltzer_nir.cfg` — per-channel instrument configuration
 - `excel_mapping.cfg` — mapping between Excel catalogue columns and internal parameter names
- `data/` — all static data files required by the simulator:
 - `waltzer_nuv_eff_area.txt`, `waltzer_vis_eff_area.txt`, `waltzer_nir_eff_area.txt` — effective area curves per channel
 - `nir_psf.txt` — NIR point spread function
 - `nuv_cross_dispersion_spread.txt` — NUV cross-dispersion spread profile
 - `nuv_spectropolarimetry.txt`, `vis_spectropolarimetry.txt` — polarization delta files for spectropolarimetry mode
 - `background.txt` — default background spectrum
 - `zod_dist.txt`, `zod_spectrum.txt` — zodiacal light distribution and spectrum
 - `stellar_param_mamjeck.txt` — Mamajek dwarf star parameter table for spectral type assignment
 - `models/` — stellar atmosphere model grids
 - `BackgroundStars/` — cached Gaia background star catalogues per target
- `input/` — target parameter files and Excel target catalogue:
 - `Targets_V10p1.xlsx` — Excel catalogue of targets and stellar parameters
 - `parameters.txt` — default parameter file loaded when no argument is given
 - `parameters/` — additional parameter files for specific targets
- `output/` — simulator output, one subdirectory per run
- `src/` — all simulator source code
- `tests/` — unit tests and snapshot tests

1.2. Configuration

All configuration files are located in `configs/`. Every parameter is documented inline in the respective file.

The simulator distinguishes between constants, configuration files and runtime parameters. The distinction is primarily based on how frequently values are expected to change and how much validation is applied when they are modified.

1.2.1. `global.cfg`

Global simulation settings: pipeline control (which channels to run), timing and observation parameters, Gaia configuration, ISM and line core emission parameters, cosmic ray settings, background configuration, and output control.

These values are configurable and validated during loading. They are expected to change occasionally between simulation runs, but are generally stable during normal operation.

For details on configuration loading, caching, and runtime access through `get_global_config()`, see Section 5.2.

1.2.2. `waltzer_nuv.cfg`, `waltzer_vis.cfg`, `waltzer_nir.cfg`

Per-channel instrument configuration: detector geometry, noise properties, effective area file, slit or PSF properties, and spread profile.

1.2.3. `excel_mapping.cfg`

The Excel mapping file decouples the Excel catalogue column names from the internal parameter names used by the simulator. Without it, column name changes in the catalogue would require modifying the source code. Instead, only the mapping file needs to be updated.

The file also separates planetary and stellar properties, so the simulator can assign parameters to the correct class (`Star` or `Planet`) without hardcoding this distinction. Finally, it defines which parameters are required. For the primary target, if any required parameter is missing after all enrichment steps — including Gaia lookup and fallback derivations — the simulator stops and reports which parameters are missing. For background stars, missing parameters are handled differently: since Gaia does not always provide a complete set of properties and not all of them can be derived, background stars with missing required parameters are silently skipped rather than stopping the simulation.

1.2.4. Constants

Constants are defined directly in the source code and are generally not intended to be modified during normal simulator operation. They typically represent implementation decisions, fixed assumptions or values that affect the behaviour of multiple pipeline stages.

Constants are not exposed through configuration files and are generally not heavily guarded against invalid modifications. Changing them assumes an understanding of the affected code and the potential consequences for the simulation pipeline.

1.2.5. `parameters.txt` / User Config

The parameter file contains the runtime inputs selected by the user for a specific simulation run. At the moment this includes the target name, the total observation length, and the exposure times for the individual channels.

The `target_name` is the critical field. It is used to select the target system from the Excel catalogue. Its properties are then read from the catalogue, and missing values can be retrieved from Gaia where implemented. Because the target name controls which system is simulated, it is strongly validated during loading and preprocessing.

The exposure times are read from `parameters.txt` and written into the channel objects during channel initialisation. They are not treated as global configuration values afterwards.

`parameters.txt` is therefore not a general simulator configuration file. It defines the concrete run setup: which target is simulated and with which exposure times.

1.3. Data Files

1.3.1. Effective Area Files

The effective area files for the three channels (`waltzer_nuv_eff_area.txt`, `waltzer_vis_eff_area.txt`, `waltzer_nir_eff_area.txt`) can be replaced with updated instrument models as long as the file format is preserved.

The file must contain the following:

- A header line `# Pixel scale: <value>` specifying the spatial pixel scale in arcseconds per pixel.
- A whitespace-delimited numeric table where the first column is wavelength in Angstrom and the last column is effective area in cm^2 . Intermediate columns are ignored.

The wavelength grid in the effective area file defines the detector pixel grid. The broadened photon flux is interpolated onto this grid, multiplied by the effective area, and multiplied by the pixel scale to produce counts per second per pixel. For spectroscopy channels (NUV and VIS), the number of wavelength rows must match

`x_pixels` in the channel configuration exactly, as each row corresponds to one detector pixel in the dispersion direction. For the NIR photometry channel no such constraint applies.

The spectral dispersion in Å per pixel is derived from the wavelength step between the first two rows of the table and stored as `spectral_dispersion_A_per_pixel` on the channel. This value is used for the Gaussian broadening of the stellar spectrum before convolution with the instrument response.

The wavelength range of each channel is entirely determined by the effective area file. There is no separate configuration for it.

1.3.2. Cross-Dispersion Spread Profile

The spread profile to use is configured per channel via `spread_profile_file`. If a file is given, the wavelength-dependent profile is loaded and used. If `spread_profile_file` is empty and `spread_half_height_pix` is set, a Gaussian fallback is used instead. If neither is configured, the simulator stops with an error.

The spread file format is a header row starting with `pixels` followed by wavelength values in Angstrom, then one row per pixel offset containing the fractional weight at each wavelength. The number of wavelength bins in the spread file does not need to match the detector pixel count — the simulator finds the nearest bin for each detector pixel and multiplies the counts by the corresponding weights to build the 2D image.

1.3.3. NIR PSF File

The PSF file is configured via `psf_file` in `waltzer_nir.cfg`. It contains a 2D numeric grid, normalised to sum to 1 after loading. The total counts are summed into a scalar, the PSF is multiplied by it, and the result is pasted onto the detector image at the source position. Portions of the PSF stamp falling outside the detector boundary are clipped.

2. Running the Simulator

Run from the repository root in all cases.

Default parameter file The simplest invocation loads `input/parameters.txt` as the default parameter file:

```
python src/cadence_simulator.py
```

Specific parameter file A specific parameter file can be passed as an argument:

```
python src/cadence_simulator.py input/parameters/parameter_001.txt
```

Batch run To run all `.txt` parameter files in a folder sequentially, use the batch runner script:

```
input/starrunner.sh input/roland
```

The script iterates over all `.txt` files in the given directory and runs the simulator for each one in sequence. Without an argument it iterates over all subdirectories of the current directory. The exit code of each run is reported.

2.1. Excel Target Catalogue

The simulator searches the `input/` directory for an Excel file at startup and loads whichever one it finds. Temporary lock files created by Excel (e.g. `~$Targets.xlsx`) are automatically ignored. Only one real Excel file may be present — placing a second one there will cause the simulator to stop with an error. The first active worksheet is used, so if the catalogue ever contains multiple worksheets, the target data must be on the first one.

The simulator looks up the target by matching the `target_name` from the parameter file against the `pl_name` column. The `pl_name` column contains planet names including the planetary suffix (e.g. `KELT-9 b`), while `target_name` in the parameter file is specified without the suffix (e.g. `KELT-9`). The simulator strips the last character from each `pl_name` entry before comparing. Target names are trimmed of whitespace, lowercased, and compared case-insensitively, so minor formatting differences including leading or trailing spaces and differing capitalisation do not cause lookup failures.

If target initialisation fails despite the name appearing correct, check the `pl_name` entry in the Excel file carefully. Excel sometimes introduces hidden characters when cell contents are copied from external sources such as websites or other documents. These are not visible in the cell but will cause the lookup to fail. To find them, paste the cell contents into a plain text editor — hidden characters will become visible there. Retyping the name manually in Excel can also resolve this.

2.2. Gaia Helper Scripts

The `src/gaiahelper/` directory contains two helper scripts that are not part of the Cadence simulation pipeline but were used to pre-fetch and organise background star data from the Gaia archive.

gaiahelper_create_background_stars.py Fetches background stars from the Gaia DR3 archive for a list of targets and saves one CSV file per target into a specified output directory. The Gaia query used is identical to the one used internally by Cadence, so the resulting CSV files are directly compatible with `data/BackgroundStars/`. Pre-fetching background stars with this script and placing the CSVs there is strongly recommended — Gaia queries during a simulation run are slow and can take a very long time for crowded fields.

generate_master_excel.py Combines all CSV files in a directory into a single Excel file for inspection. Useful for getting an overview of all pre-fetched background stars across targets.

2.3. Gaia Lookups

Gaia is queried in two situations during a simulation run.

Target star If required stellar parameters are missing from the Excel catalogue after loading, the simulator queries Gaia DR3 for the target and fetches all available parameters. Only those missing from the Excel catalogue are then merged into the parameter set — Excel values always take precedence and are never overwritten by Gaia.

Background stars If no CSV file for the target is found in `data/BackgroundStars/`, the simulator queries Gaia to fetch background stars at runtime. The resulting CSV is saved to the output directory and can be moved to `data/BackgroundStars/` for subsequent runs. It is strongly recommended to pre-fetch background stars using `gaiahelper_create_background_stars.py` before running the simulator. Gaia queries are slow, unreliable, and can take a very long time for crowded fields.

The cone search radius is controlled by `gaia_conesearch_radius_arcsec` in `global.cfg` (default 500 arcsec). The cached CSV contains only stars within the radius that was set when it was fetched. If a CSV is saved to `data/BackgroundStars/` with a small radius and the NIR channel covers a larger field of view, background stars outside that radius will be missing from the simulation. Always ensure the cone search radius is at least as large as the largest channel field of view when pre-fetching background stars.

Gaia queries can be run in asynchronous or synchronous mode via `GAIA_USE_ASYNC_JOBS` in `global.cfg`. In practice, neither mode is reliably fast or stable — if one fails or times out, try switching to the other.

Gaia Query The Gaia query fetches the following parameters: `source_id`, `right_ascension`, `declination`, `parallax`, `gaia_magnitude`, `effective_temperature`, `distance`, `radius`, `mass`, `metallicity`, and `surface_gravity`. These are defined in `GAIA_PROPERTIES` in `src/loaders/load_gaia.py`, analogous to the Excel mapping file. The query joins `gaia_source`, `astrophysical_parameters`, and `astrophysical_parameters_supp` from Gaia DR3, and uses `COALESCE` to retrieve as complete a set of parameters as possible for each source. The priority order in each `COALESCE` expression prefers observed and directly derived quantities over model-based ones, so that background stars with only partial Gaia coverage are still included where possible. These parameter names are used throughout the codebase and mapped directly to the internal parameter dictionary and the `Star` class. Changing them would require updates in many places and is strongly discouraged. To add a new Gaia parameter, add it to `GAIA_PROPERTIES`, add the corresponding alias to the query, and extend the `Star` class and any downstream code that uses it.

Target Star Lookup To query Gaia for the target star, the simulator first resolves the target name via SIMBAD to obtain a Gaia DR3 source ID. If SIMBAD fails or returns no Gaia ID, a small cone search of 6 arcsec around the RA/Dec from the Excel catalogue is used to find the closest source. Once the source ID is known, the full parameter query is run against that specific source ID.

The resolved source ID is also used when fetching background stars — the target itself is identified by source ID and excluded from the background star catalogue. If the source ID cannot be resolved, the target star may end up included as a background star, which has happened in cases where the RA/Dec in the Excel catalogue differed by more than 6 arcsec from the Gaia position. If background star results look suspicious, verifying the source ID resolution in the log file is recommended.

Parameter Enrichment After loading from the Excel catalogue and optionally merging Gaia parameters, the simulator applies a series of fallback derivations to complete the parameter set before initialising the target.

- **Distance** — derived from Gaia parallax as $d = 1000/\varpi$ if missing.
- **Radius** — estimated from Gaia G magnitude, distance, and effective temperature via the Stefan–Boltzmann relation if missing.
- **Spectral type** — inferred from effective temperature using the Mamajek dwarf star table if missing.
- $\log R'_{\text{HK}}$ — assigned from a temperature threshold defined in `global.cfg` (`log_r_teff_threshold`, `log_r_hot_value`, `log_r_cool_value`) if missing.

Effective temperature and mass are not derived. If effective temperature is missing after Excel and Gaia lookup, the simulator stops for the target star, as it is required for all downstream processing. Mass is optional and stays `None` if not provided.

3. Tests

The test suite consists of unit tests and snapshot tests. Snapshot tests compare freshly computed simulator output against previously saved reference arrays produced by a pipeline run. They guard against unintended numerical changes when the pipeline is modified. If a change intentionally alters numerical output, the snapshots must be regenerated.

Snapshot tests are the most important test category, as they freeze the pipeline output end-to-end. Effective area files and all configuration files are copied into the snapshot directory (as npz) alongside the reference arrays, so changes to those files do not break the tests. Only implementation changes require regenerating snapshots — which is correct behaviour, since a changed implementation may either fix a bug or introduce one, and the snapshots should reflect the intended output.

3.1. `test_flux_pipeline.py`

This suite covers the spectral modelling steps from stellar atmosphere model through to photon flux density at Earth. Each pipeline stage is tested independently: stellar intensity to luminosity conversion, chromospheric line core emission, interstellar medium absorption, geometric dilution to flux at Earth, dereddening, and conversion to photon flux density. Tests are run for WASP-69, WASP-189, and HD 2685. Snapshots are plain text files written by the simulator when `write_intermediate_arrays = 1` is set in `configs/global.cfg`.

3.2. `test_detector_pipeline.py`

This suite covers the instrument simulation steps from photon flux density through convolution with the instrument response, spreading to 2D detector images, and science frame assembly. All three channels (NUV, VIS, NIR) are tested for KELT-9 and HD 2685. Snapshots are `.npz` files written by the simulator when `write_intermediate_arrays = 1`.

3.3. Regenerating Snapshots

1. In `configs/global.cfg`, set `write_intermediate_arrays = 1` and `orbit_revolutions = 0.1` (only the intermediate array files are needed; a full orbit is not required).
2. Run the simulator for the targets relevant to the changed pipeline stage.
For `test_flux_pipeline.py`: WASP-69, WASP-189, and HD 2685.
For `test_detector_pipeline.py`: KELT-9 and HD 2685.
3. Copy all `.txt` snapshot files (used by `test_flux_pipeline.py`) and all `.npz` snapshot files (used by `test_detector_pipeline.py`) from each output directory into `tests/snapshot_tests/snapshots/`.
The `.npz` binary format is used for detector snapshots because detector images are large and loading them as text would make the tests prohibitively slow.
4. Run the simulator for the targets relevant to the changed pipeline stage. For `test_flux_pipeline.py`: WASP-69, WASP-189, and HD 2685. For `test_detector_pipeline.py`: KELT-9 and HD 2685.
5. Verify the full test suite passes: `python -m pytest tests/`
6. Reset `write_intermediate_arrays = 0` and `orbit_revolutions` to the original value for normal simulation runs.

4. Extending the Simulator / Performance Considerations

4.1. Adding Star and Planet Parameters

To add a new stellar parameter to the simulation, the following steps are required:

1. Add the parameter as a column to the Excel target catalogue.
2. Add the mapping from the Excel column name to the internal parameter name in the `[stars]` section of `excel_mapping.cfg`. If the parameter is required for the simulation to run, add it to `[required_stellar_parameters]` as well.
3. Add the parameter as a field to the `Star` class in `src/domain/star.py`.
4. If the parameter can be fetched from Gaia, add it to `GAIA_PROPERTIES` in `src/loaders/load_gaia.py` and add the corresponding alias to the query in `_gaia_select_joined_base()`.
5. If a fallback derivation is possible when the parameter is missing, implement it in `src/loaders/load_stellar_and_planetary_properties.py`.

For planetary parameters, the process is the same but using the `[planets]` section of `excel_mapping.cfg` and the `Planet` class. Note that the planet is currently only used to store properties loaded from the Excel catalogue and is not yet used in any simulation calculations.

4.2. Extending Channel and Global or User Configuration

Adding a channel parameter Channel parameters are defined in the `Channel` base class in `src/configs/channel_config.py`. `SpectroscopyChannel` and `PhotometryChannel` both inherit from it. Decide first whether the new parameter applies to all channels (`Channel`), spectroscopy only (`SpectroscopyChannel`), or photometry only (`PhotometryChannel`), and add the field to the appropriate class.

Note that `Channel` is never instantiated directly — only `SpectroscopyChannel` and `PhotometryChannel` are. This means even if the parameter is defined on `Channel`, it must be passed when constructing a `SpectroscopyChannel` or `PhotometryChannel` instance in `src/loaders/load_channel.py`, where the channel config files are parsed and the channel objects are created.

Adding a global config parameter Global configuration parameters are defined in the `GlobalConfig` dataclass in `src/configs/global_config.py`. To add a new parameter, add it to the dataclass, parse it from the config file in `_read_global_cfg()`, and pass it to the `GlobalConfig` constructor. `load_global_config()` must be called exactly once at startup — it reads the config file and caches the result. Everywhere else in the codebase, use `get_global_config()` to access it. Never call `load_global_config()` more than once, as this would create a second instance and the cached singleton would not be updated.

User config and run context The user config (`UserConfig`) is loaded once at startup and passed through explicitly rather than accessed globally. It contains the target name and exposure times per channel. The target name is stored in the `RunContext` after initialisation and passed through the pipeline from there. Exposure times are read from the user config and written directly into the channel objects during channel initialisation, so they do not need to be accessed separately afterwards. In practice, the user config is consumed during startup and the `RunContext` carries the relevant information for the rest of the run.

`RunContext` is a small frozen dataclass containing the target name, the output directory for the current run, and the run timestamp. It is created once at startup and passed to all pipeline stages that need to write output or log target-specific information.

4.3. Adding Additional ISM Absorption Lines

Additional ISM absorption lines can be added by extending the existing ISM line and absorber definitions.

For implementation details and required updates to the wavelength-window optimisation, see Section [5.1.2](#).

5. Caches and Performance Improvements

5.1. ISM Absorption Optimisation / Voigt Profile Cache

The ISM absorption calculation was identified as a performance bottleneck because the Voigt profiles were evaluated over the full wavelength grid. The implemented ISM model only affects the wavelength region around the Mg II, Mg I and Fe II line centres. The calculation was therefore restricted to this region + margin of 200Å.

```
line_centers = [
    lineMg1['wave'],
    lineMg2['wave'],
    lineMgI['wave'],
    lineFeII['wave']
]

wl_min = min(line_centers) - SPECTRAL_WINDOW_MARGIN_A
wl_max = max(line_centers) + SPECTRAL_WINDOW_MARGIN_A

wl = flux[:, 0]
mask = (wl >= wl_min) & (wl <= wl_max)

if not np.any(mask):
    return flux

wl_ism = wl[mask]
```

The thought process behind this change was that the implemented ISM absorption only contains these specific transitions. Outside the selected wavelength window, the transmission of the implemented model is treated as one. Leaving the flux unchanged outside the mask therefore gives the same result as multiplying it by a transmission of one.

Only the affected part of the flux array is modified:

```
flux_absorption = n_flux.copy()
flux_absorption[mask] = ISM * n_flux[mask]
```

This keeps the output identical to the full-grid calculation for the implemented ISM model, while reducing the number of wavelength samples passed through the Voigt profile calculation.

If additional ISM species are added later, their line centres must also be included in the window definition. Otherwise the mask could exclude wavelengths where newly implemented absorption should be applied.

5.1.1. Voigt Profile Cache

The Voigt profile calculation was also cached. The cache key contains only the quantities that determine the shape of the Voigt profile:

```
cache_key = (
    wavelength.tobytes(),
    float(absorber["B"]),
    float(absorber["Z"]),
    float(line["wave"]),
    float(line["gamma"]),
)
```

The cache stores the Voigt profile shape, not the final optical depth. The profile shape is determined by the wavelength grid (wavelength), the Doppler broadening parameter (B), the absorber velocity shift (Z), the line centre (wave) and the damping parameter (gamma). The absorber column density (N) is applied only after the cached profile has been calculated, when the optical depth is computed. It is therefore not part of the cache key. Since N is not used during the Voigt profile calculation, it must not be included in the cache key.

The cache implementation is independent of the number of supported ISM transitions. Newly added lines automatically generate separate cache entries through their line centre and damping parameters. **No cache modifications are required when adding additional ISM lines.**

The cache implementation only needs to be revisited if additional parameters are introduced that modify the Voigt profile shape itself. Simply adding new ISM transitions does not require changes to the cache implementation.

5.1.2. Adding Another ISM Absorption Line

Additional ISM absorption lines can be added to the existing line core absorption model, but all parts of the calculation must be updated consistently.

First, define the additional line parameters. The line centre must be added to the list used for the ISM wavelength window:

```
line_centers = [  
    lineMg1['wave'],  
    lineMg2['wave'],  
    lineMgI['wave'],  
    lineFeII['wave'],  
    lineNew['wave']  
]
```

The corresponding absorber and line definition must then be passed to the Voigt profile calculation:

```
ISMNew = voigtq(wl_ism, absorberNew, lineNew)
```

The new transmission must also be included in the total ISM transmission:

```
ISM = ISMMg21 * ISMMg22 * ISMMg1 * ISMFe2 * ISMNew
```

The important point is that every newly implemented ISM line must be included in the wavelength window. If the line centre is not added to `line_centers`, the mask may exclude the relevant wavelength region and the new absorption line will not be applied.

The Voigt profile cache does **not** need to be changed when adding a normal line that uses the existing profile calculation.

5.2. Global Configuration Cache

The global configuration must be loaded exactly once during program startup and is subsequently reused through a module-level cache.

```
def load_global_config(path: Path) -> GlobalConfig:  
    global _GLOBAL  
    if _GLOBAL is None:  
        _GLOBAL = _read_global_cfg(path)  
    return _GLOBAL
```

Configuration files must not be reloaded during normal program execution. Components requiring configuration values must access the already loaded configuration object through:

```
cfg = get_global_config()
```

All configuration access throughout the codebase must use this mechanism. This guarantees that every component operates on the same configuration instance and avoids repeated configuration file parsing and validation.

5.3. User Configuration Cache

see above Section 5.2. Same style, but usually i passed the user config around, since it was not often used. However, it is later more often used, you can use it similarly to `globalConfig()`.

5.4. Stellar Model Caches

Stellar model loading is centralised in `load_model_for_temperature()`. All code requiring stellar model data must use this function and must not load model files directly.

The model loading system uses three caches:

```
_MODEL_CACHE = {}  
_AVAILABLE_MODELS = None  
_CUT_MODEL_CACHE = {}
```

Available Model Cache The available model directories are scanned only once. During the first call, the contents of `data/models` are inspected and the result is stored in `_AVAILABLE_MODELS`. Subsequent calls reuse the cached directory list and do not perform another filesystem scan.

Model Cache The raw stellar model data loaded from disk is stored in: `_MODEL_CACHE`. The cache key is the full model file path. As a result, the cache is independent of the model selection logic. Changes to the available stellar models or temperature matching algorithm do not require modifications to the cache implementation, provided that each model continues to be identified by a unique file path.

Cut Model Cache The wavelength-trimmed model data is stored in: `_CUT_MODEL_CACHE`. After loading, the model spectrum is restricted to the wavelength range obtained via the cdfs of `nuv`, `vis` and `nir` required by the simulation:

```
cut_model_data = _cut_model_wavelength_range(full_model_data, wl_min_A, wl_max_A)
```

The resulting array contains the wavelength grid and the corresponding model values within the selected wavelength range. This processed array is stored in the cache and reused by later requests.

The cache therefore avoids repeated wavelength trimming and repeated model file loading.

The cached arrays are marked as read-only:

```
cut_model_data.setflags(write=False)
```

Code receiving model data from `load_model_for_temperature()` must treat the returned array as immutable. Any modification requires creating a copy first, which we already do at the beginning of the flux pipeline.

This cache is especially important for simulations with many background stars, where the same stellar model can be requested repeatedly.

5.5. Unred Curve Cache

The extinction curve construction in `cute_unred` is cached through: `_UNRED_CURVE_CACHE = {}`. The cache stores the wavelength-dependent extinction curve. The cache key consists of:

```
cache_key = (wave.tobytes(), float(R_V), bool(LMC2), bool(AVG_LMC))
```

The cache therefore depends only on the wavelength grid and the selected extinction-law parameters. The extinction curve is independent of both the input flux and the colour excess $E(B-V)$.

The expensive spline-based extinction curve construction is performed only once for a given wavelength grid and extinction-law configuration. Subsequent calls reuse the cached extinction curve and only apply the final $E(B-V)$ scaling.

The cache key includes all quantities used during extinction-curve construction. Changes to stellar models, wavelength sampling, or extinction-law configuration automatically generate separate cache entries and therefore do not require modifications to the cache implementation.